

Call for Semester Project Proposals (Algorithms)

Overview

For the final project, each team will explore a **recent, high-quality research contribution in algorithms** and connect it to practical implementation and evaluation. This project is designed to bridge the gap between theory and practice: you will read a research paper, understand the underlying algorithmic ideas, and either **implement and test a contribution** from the paper, or perform a **comparative study of multiple modern algorithm design techniques** on a real-world problem.

Team Formation

- **Group size:** 2–3 students per team
- You are responsible for forming your own teams and coordinating your work.

Grading Weightage

- **Total project weight:** 20% of the course grade
- **Proposal weight:** 5% (part of the total 20%)

Project Options (Choose ONE)

Option A: Paper-Based Implementation

Your team will select a **recent research paper (published within the last 5 years)** from a **high-quality algorithms conference** (see the candidate conference list at the end). The chosen paper must have a **clear algorithmic component** that can be **implemented** and **tested**.

Your implementation should reproduce a key idea, component, optimization, or improvement proposed in the paper (not necessarily the entire system), along with a reasonable evaluation.

Option B: Comparative Analysis on a Real-World Problem

Your team may choose a **real-world problem** and perform a **comparative analysis** of **different algorithm design techniques**, with emphasis on **recent techniques**.

- You must clearly define the problem setting, constraints, datasets/inputs, and evaluation criteria.
- The comparison must include **2–3 approaches** (minimum) and should not be limited to only classic baselines.
- For each selected approach/technique, you must include **supporting references to the research literature** (preferably **within the last 5 years**).

Proposal Submission (ACL L^AT_EX, 2–3 pages)

Submit a **2–3 page proposal** written in **ACL L^AT_EX format** (ACL-style template). Your proposal must include:

1. Paper / Problem Information

- **Option A (Paper-Based):** Paper title; authors; conference; year of publication.
- **Option B (Comparative Analysis):** Problem title and description; the techniques/approaches you plan to compare (**at least 2–3**); and **citations** including **at least one relevant paper reference per technique** (preferably within the last 5 years).

2. Brief Summary

- A concise description of the problem being addressed.
- The main contribution(s) you plan to implement or evaluate.
- Expected inputs/outputs.

3. Feasibility + Relevance Justification

- Why your choice is **relevant** to algorithmic design and analysis.
- Why it is **feasible** to complete within the semester.
- A clear scope statement (what you will implement/compare, and what you will **not** do).

4. Implementation / Experiment Plan

- High-level plan of what you will build and how you will test it.
- Baselines and metrics you will use.
- Any assumptions, limitations, or risks.

5. Resources Declaration

- Existing implementation / code repository (if available).
- Dataset, benchmark instances, or instance generator.
- libraries/tools that you anticipate needing.

Important: A proposal is expected to show genuine initial investigation (paper read-through, problem understanding, and a realistic plan). Vague proposals may be returned for revision.

Sample Problems

Following are some suggested computational problems for your projects. Your instructor may share further problems with you, so you must meet your instructor at your earliest when the call for proposals is shared.

1. Shortest path problem
2. Minimum spanning tree problem
3. String matching problem

4. Traveling salesperson problem
5. Matrix multiplication problem
6. Pairwise sequence matching problem
7. Maximum flow/Minimum Capacity in a network flow problem
8. Fast Fourier Transform
9. Data Encryption algorithms

Rubric (5 Points Total)

1. **Paper Selection & Relevance (1 point)**
2. **Problem Understanding & Technical Summary (1 point)**
3. **Implementation / Experimental Plan (1 point)**
4. **Evaluation Design & Fair Comparison (1 point)**
5. **Clarity, Feasibility, & Risk Management (1 point)**

Candidate Conferences (Algorithms)

Theoretical Computer Science & Algorithms

- **STOC** — ACM Symposium on Theory of Computing
- **FOCS** — IEEE Symposium on Foundations of Computer Science
- **SODA** — ACM-SIAM Symposium on Discrete Algorithms
- **ICALP** — International Colloquium on Automata, Languages, and Programming
- **ESA** — European Symposium on Algorithms
- **APPROX** — Approximation Algorithms for Combinatorial Optimization Problems
- **RANDOM** — International Workshop on Randomization and Computation
- **ITCS** — Innovations in Theoretical Computer Science

Graph Algorithms & Data Structures

- **WG** — Workshop on Graph-Theoretic Concepts in Computer Science
- **IWOCA** — International Workshop on Combinatorial Algorithms

Optimization & Combinatorial Algorithms

- **IPCO** — Integer Programming and Combinatorial Optimization
- **MPS** — Mathematical Programming Symposium

Related Conferences in Theoretical CS & Applied Algorithms

- **COLT** — Conference on Learning Theory
- **LIPIcs** — Leibniz International Proceedings in Informatics (proceedings venue; includes multiple algorithms/theory conferences)
- **SPAA** — ACM Symposium on Parallelism in Algorithms and Architectures